Google Developers



Debajit Mallick

Software Engineer @Ergeon,
Organizer @GDG Siliguri & @React Siliguri



who am I?

- 37+ talks on topics like Web Performance, Frontend, Firebase and many more.
- LinkedIn 2024 Top Voice for Web Development.
- Mentor and Judge of Hack4Bengal 3.0 and 2.0 Hackathon.
- Mentor of team OrganiCod3rs, the Winner of Smart India Hackathon 2022, Software Edition.
- Mentor of GirlScript Summer of Code 2023.
- Member of Team Delenitors, Smart India Hackathon 2020 Winner, Software Edition.
- Top Contributor of GirlScript Winter of Code 2021.
- Top Contributor of JGEC Winter of Code 2020.

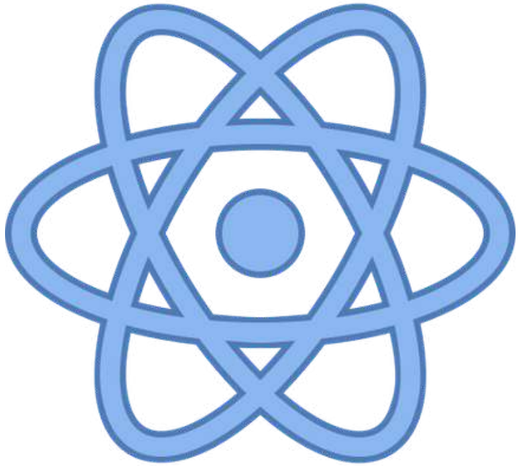



CHALIYE
SHURU
KARTE HAI

Google Developers



Gemini



Topic

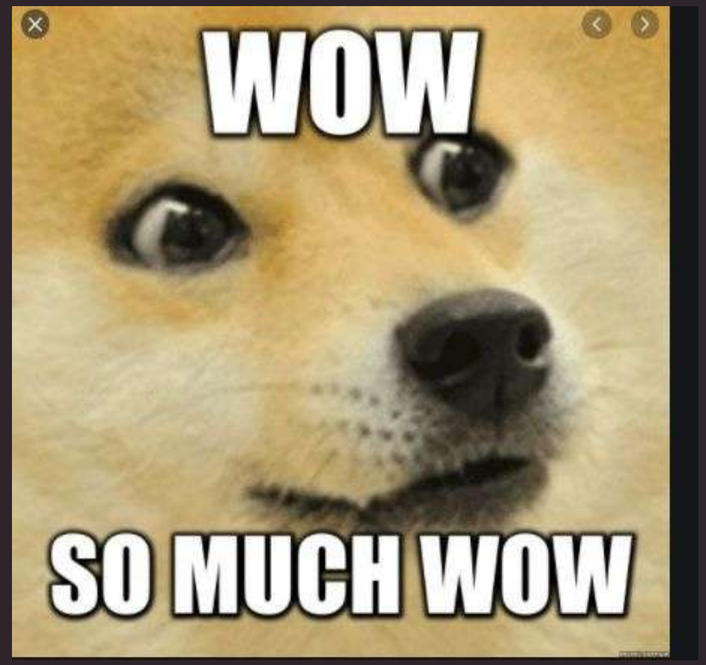
Building Production Apps using Firebase and AI



How can I build a full-stack AI-powered application without backend



The solution



Topics

- What is Firebase?
- What is Firebase Realtime Database & Firebase Auth?
- Developing a full-stack app using React, Firebase & Gemini
- Memes




yeh sab kya tha

What is Firebase Realtime Database?

The Firebase Realtime Database is a cloud-hosted database. Data is stored as JSON and synchronized in realtime to every connected client.




SONY
SAB
SONY
SAB

What is Firebase Auth?

Firebase Auth is a backend service that provides secure user authentication for applications, simplifying the process of managing user sign-ups, sign-ins, and logouts.



What is Gemini?

Gemini is Google's family of multimodal AI models that power a range of products, from the Gemini chatbot to its integration into Google Workspace (like Gmail and Docs) and the Gemini mobile app



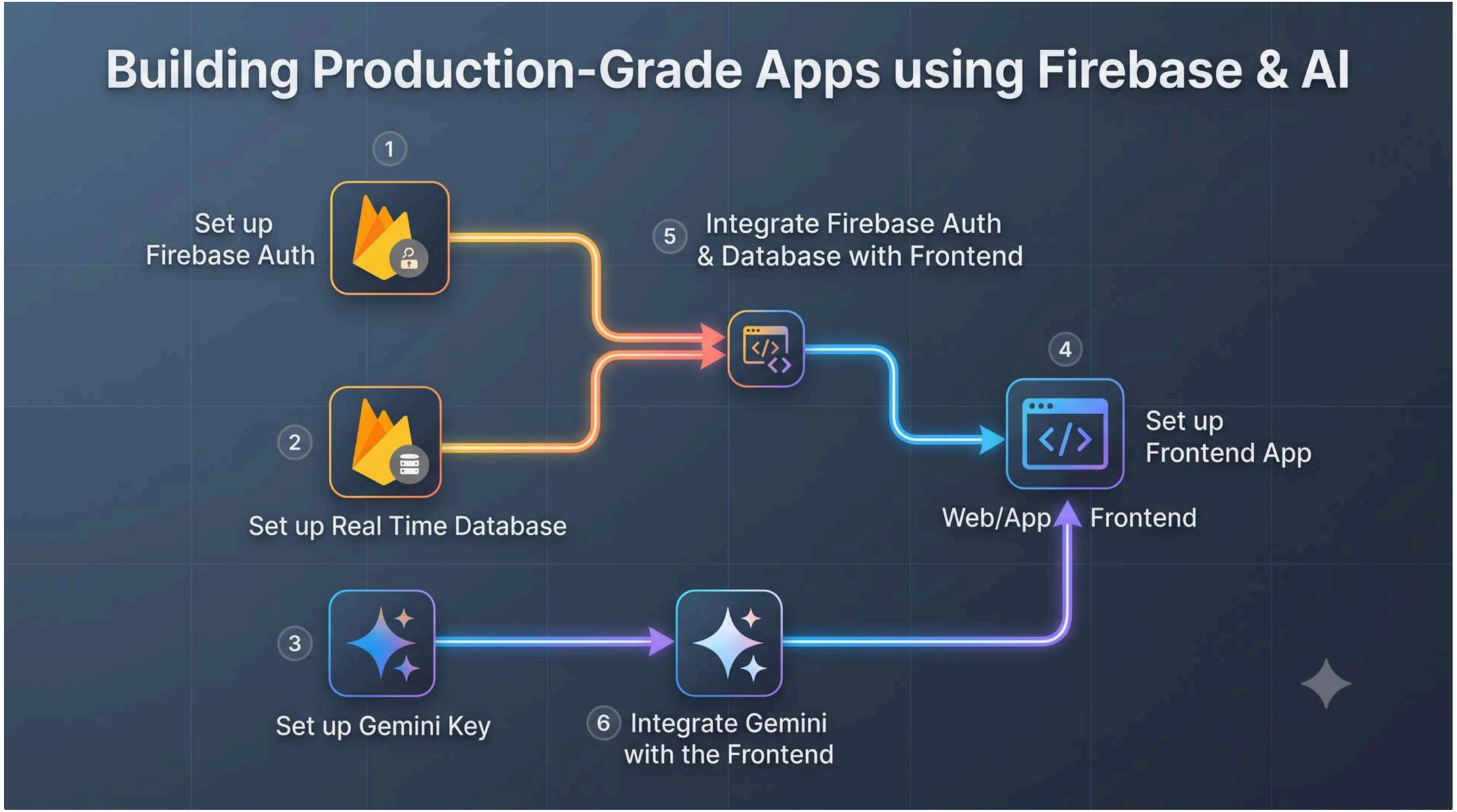
Steps to Build the App

- Set up Firebase Auth
- Set up Real Time Database
- Set up Gemini Key
- Set up Frontend App
- Integrate Firebase Auth and Database with Frontend
- Integrate Gemini with the Frontend




Abhi maza ayega na bhidu

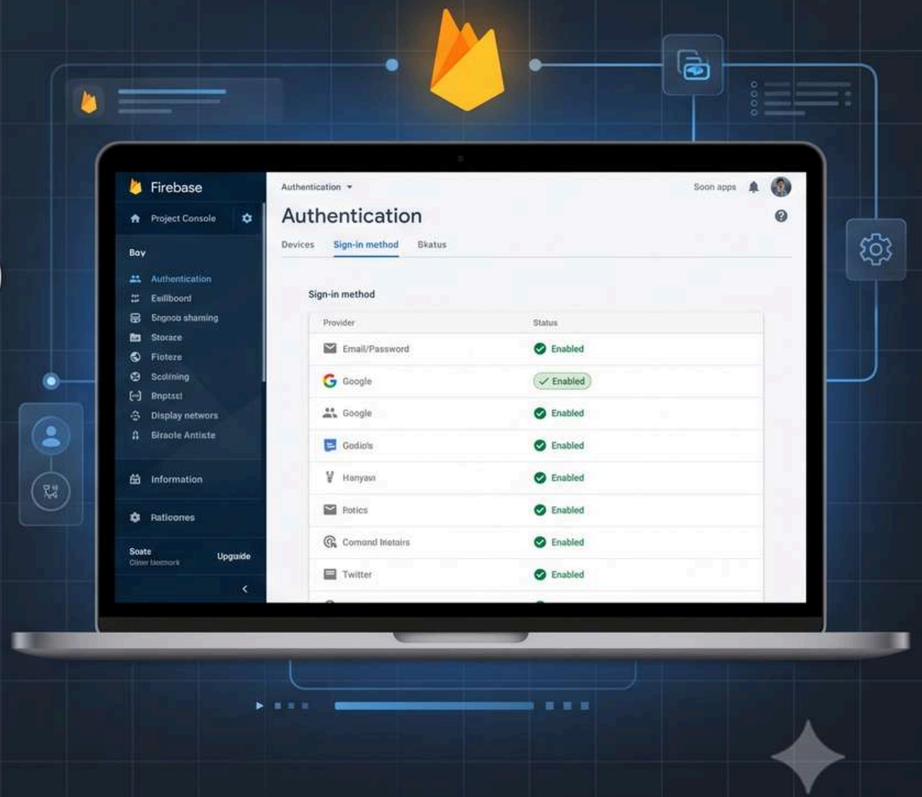
Building Production-Grade Apps using Firebase & AI



Step 1: Set up Firebase Auth

1. Create & Configure Firebase Auth

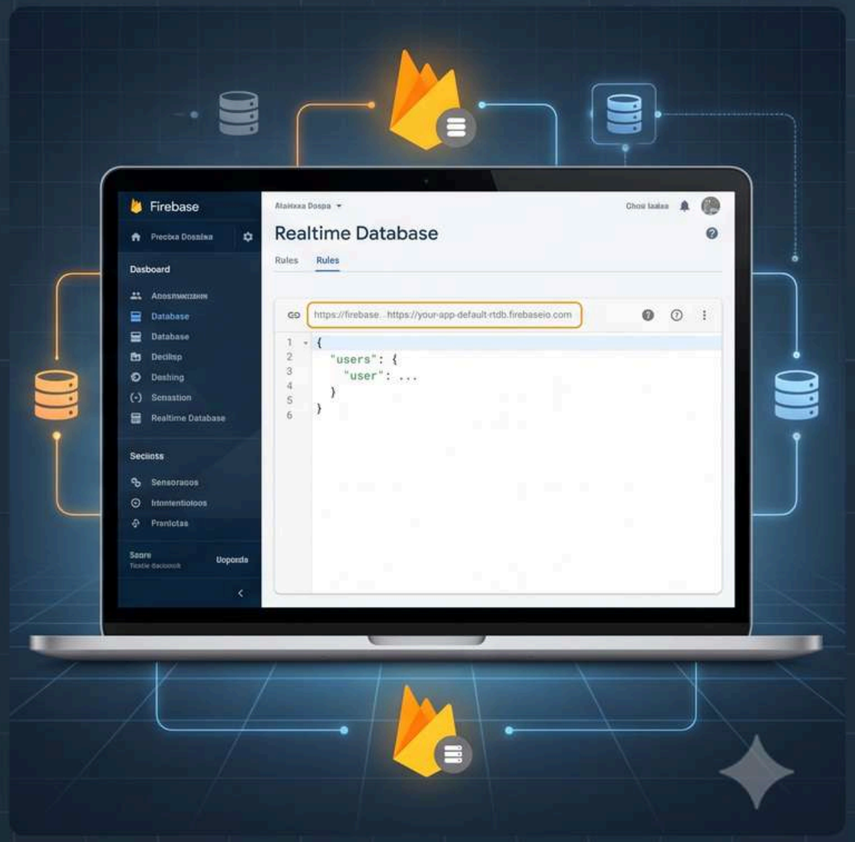
- Create a new project in the Firebase Console (console.firebase.google.com)
- Navigate to the Build menu and select **Authentication**
- Click '**Get Started**' and enable the desired Sign-in providers (e.g., Email/Password, Google)
- Download the `firebase-config` file for your app initialization



Step 2: Set up Realtime Database

2. Set up Firebase Realtime Database

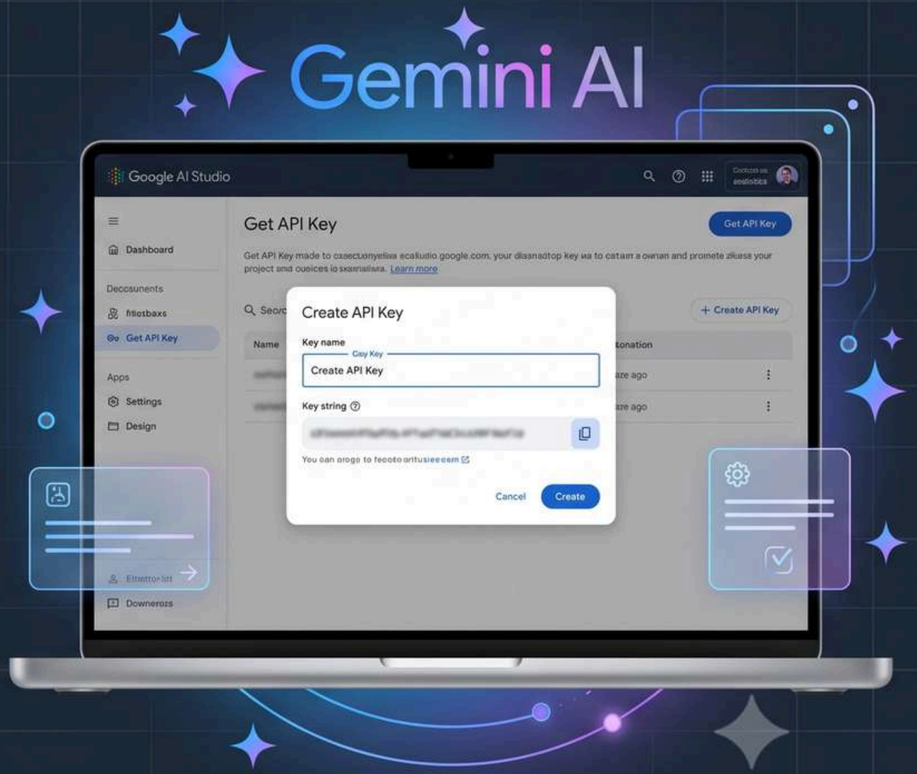
- In the Firebase Console sidebar, navigate to "Build" > "Realtime Database"
- Click "Create Database", choose your database location, and click "Next"
- Configure security rules (start in "test mode" for development, but switch to strict rules for production)
- Take note of your database URL (e.g., https://your-app-default-rtdb.firebaseio.com) for frontend connection



Step 3: Set up Gemini Key

3. Generate and Secure Gemini API Key

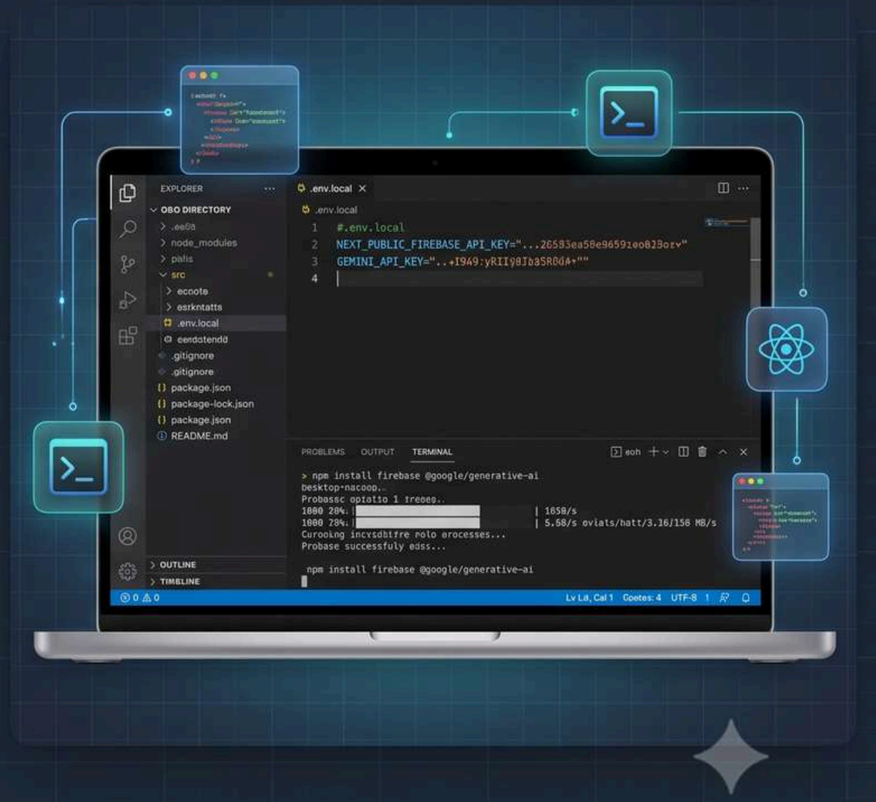
- Go to Google AI Studio (aistudio.google.com)
- Click “Create API key” or “Get API key”
- Select your Google Cloud project (or create a new one) to generate the key
- Copy the generated API key **immediately** and **store it securely** (e.g., in a `.env` file)
- **Warning:** Never hardcode or expose this key in public repositories



Step 4: Set up Frontend App

4. Create and Configure Frontend App

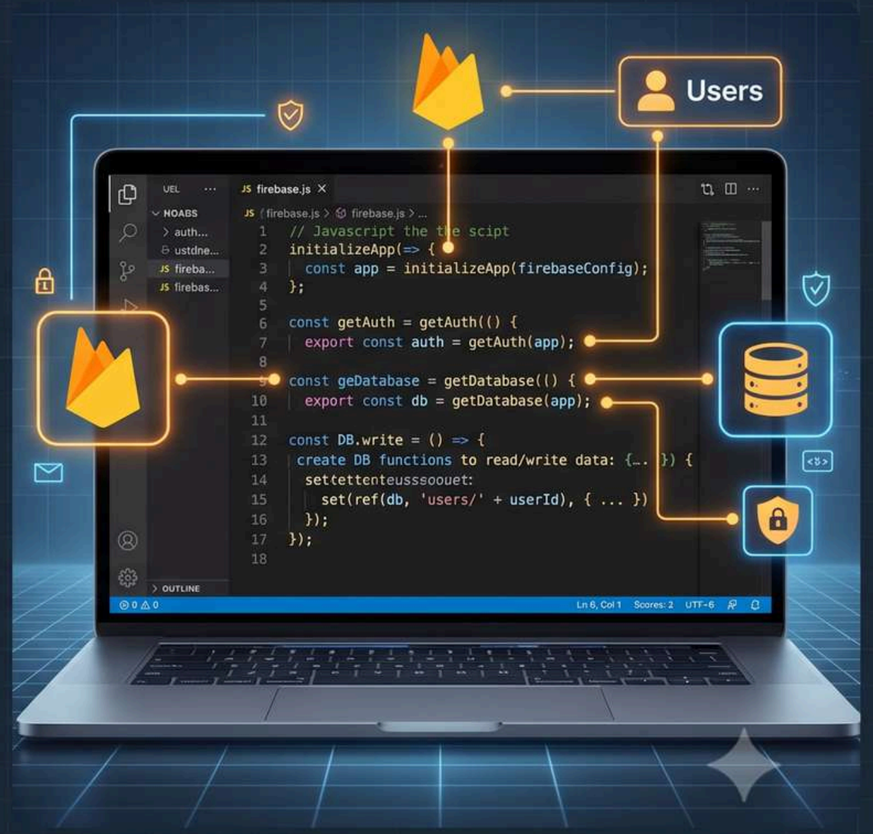
- Initialize your project (e.g., `npx create-next-app@latest` or choose your favorite framework)
- Install the required dependencies: `npm install firebase @google/generative-ai`
- Configure environment variables in a secure `.env.local` file (never commit these)
- Set up the basic folder structure, UI layout, and import/initialization files



Step 5: Integrate Firebase Auth and Database

5. Integrate Firebase Auth & Database

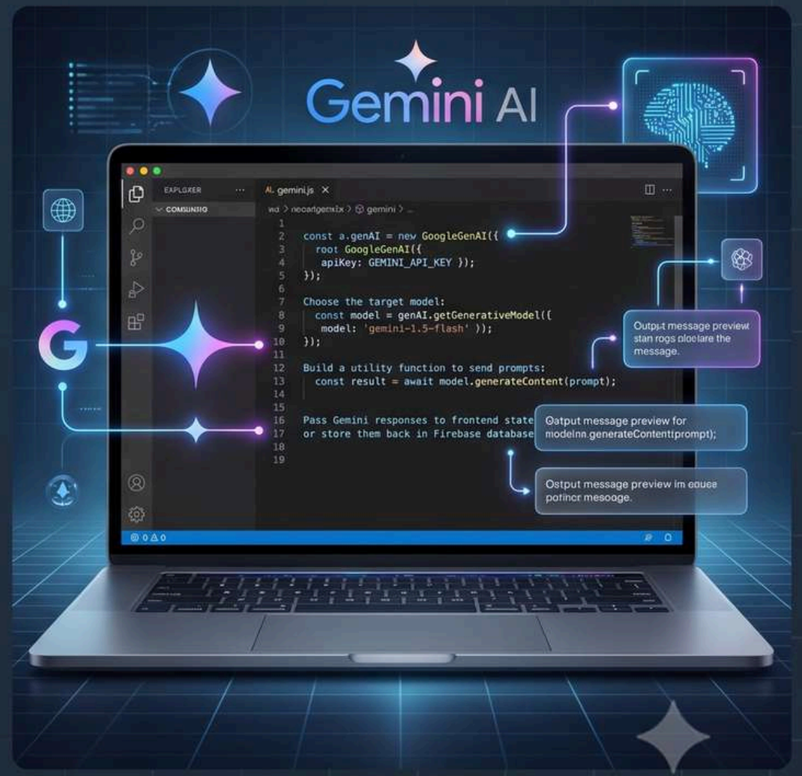
- Create a file (e.g., `firebase.js`) and initialize Firebase
- Code: `const app = initializeApp(firebaseConfig);`
- Export initialized services: `export const auth = getAuth(app); and `export const db = getDatabase(app);`
- Create a frontend context or utility to handle ‘onAuthStateChanged’ and Auth triggers
- Create DB functions to read/write data: `set(ref(db, 'users/' + userId), { ... })`



Step 6: Integrate Gemini with Frontend

6. Integrate Gemini AI with the Frontend

- Create an AI configuration file (e.g., `gemini.js` or API route)
- Initialize Gemini client: `const genAI = new GoogleGenAI({ apiKey: GEMINI_API_KEY });`
- Choose the target model: `const model = genAI.**getGenerativeModel({** model: **'gemini-1.5-flash' });**`
- Build a utility function to send prompts: `const result = await model.generateContent(prompt);`
- Pass Gemini responses to frontend state or store them back in Firebase database




Mai... Mereko sab aata hai...
Mai Expert Hu



Google Developers

Thank You

x.com/MallickDebajit

linkedin.com/in/debajit-mallick/

github.com/debajit13